Typing Race Simulator

Object Oriented Programming Project - ECS414U

Part I: Textual Simulation - Report

Author: Bahooz Mustafa
250265831

Module: ECS414U — Object Oriented Programming

Date: 2026

Table of Contents

1. Introduction 3
2. Typist Class - Design and Implementation 4
   2.1 Fields 4
   2.2 Constructor 4
   2.3 Encapsulation 5
      slideBack(int amount) 6
      setAccuracy(double newAccuracy) 6
   2.4 Accessors (Getters) 7
   2.5 Mutators (Setters) 8
   2.6 Action Methods 8
3. TypingRace Class - Bug Analysis and Fixes 10
4. Testing and Validation 14
   Terminal Output (Test Run) 14
5. Conclusion 17

1. Introduction

This report documents the completion and improvement of the Typing Race Simulator, an Object Oriented Programming project for ECS414U. The project was inherited from a fictional predecessor, Ty Posaurus, who left the codebase in an incomplete and buggy state.

The simulation models a typing race between three competitors. Each typist advances through a passage of text one character at a time, but may slide backwards on mistypes or freeze entirely when burnt out. The first typist to reach the end of the passage wins.

The work completed in Part I includes:

- Full implementation of the Typist class from an empty skeleton.
- Identification and correction of 7 bugs in the TypingRace class.
- A comprehensive test suite covering all required boundary cases.

2. Typist Class - Design and Implementation

The Typist class represents a single competitor in the race. It was provided as a skeleton with method signatures but no fields, no constructor body, and no method implementations. The following sections explain each design decision.

2.1 Fields

The class requires exactly six fields, each declared **private** for encapsulation:

Field	Type	Purpose
name	String	Display name of the typist (e.g. "TURBOFINGERS")
symbol	char	Single Unicode character shown on the race track (e.g.
accuracy	double	Typing accuracy in range [0.0, 1.0]
progress	int	Characters typed so far; can go up and down but never
burntOut	boolean	True while the typist is frozen in a burnout state
burnoutTurnsRemaining	int	Countdown of turns left in burnout (0 when not burnt out

'①')
below zero
)

2.2 Constructor

The constructor accepts three parameters matching the assignment specification: a character symbol, a String name, and a double accuracy. All other fields are initialised to their default starting values (progress = 0, burntOut = false, burnoutTurnsRemaining = 0).

The accuracy parameter is set through **setAccuracy()** rather than direct assignment. This ensures that even values passed at construction time are clamped to the valid [0.0, 1.0] range, preventing invalid state from the very start.

```
public Typist(char typistSymbol, String typistName, double typistAccuracy)
{
    symbol   = typistSymbol;
    name     = typistName;
```

```
        setAccuracy(typistAccuracy);   // uses setter so clamping applies at construction
        progress                    = 0;
        burntOut                    =  false;
        burnoutTurnsRemaining = 0;
    }
```

2.3 Encapsulation

Encapsulation is the OOP principle of hiding a class's internal data behind a controlled interface. In the Typist class this is achieved by declaring all six fields as private. Outside code including TypingRace cannot read or write these fields directly; it must use the public methods provided.

> **Why does this matter?** Without encapsulation, TypingRace could set progress to -50 or accuracy to 99.0 by accident. With private fields and validated setters, these impossible states simply cannot occur.

The two methods requiring particular attention are:

slideBack(int amount)

slideBack() is a mutator that decreases progress. The critical validation here is the zero-floor: after subtracting *amount*, if progress has gone below zero it is immediately clamped back to zero. Without this guard a typist's position could become negative, causing the race display to break and the lane marker to appear left of the starting line.

```
public void slideBack(int amount)
{
    progress -= amount; if
    (progress < 0)
    {
        progress = 0;    // cannot go behind the starting line
    }
}
```

setAccuracy(double newAccuracy)

setAccuracy() clamps its input to the range [0.0, 1.0]. Values below 0.0 are set to 0.0; values above 1.0 are set to 1.0. This is essential because TypingRace multiplies accuracy by random numbers to decide typing and burnout events — an out-of-range accuracy would distort all of these calculations.

```
public void setAccuracy(double newAccuracy)
{
    if (newAccuracy < 0.0)        accuracy = 0.0;
    else if (newAccuracy > 1.0) accuracy = 1.0;
    else                          accuracy = newAccuracy;
}
```

2.4 Accessors (Getters)

Accessors provide read-only access to private fields. They return the stored value without allowing modification. The Typist class provides six accessors:

Method	Returns	Notes
getAccuracy()	double	The accuracy rating in [0.0, 1.0]
getProgress()	int	Current position along the passage (>= 0)
getName()	String	The typist's display name
getSymbol()	char	The unicode character shown on the race track
getBurnoutTurnsRemaining()	int	Turns of burnout left (0 = not burnt out)
isBurntOut()	boolean	True if currently in a burnout state

2.5 Mutators (Setters)

Mutators allow controlled modification of private fields. The Typist class provides two simple setters plus several action methods that modify state:

- **setAccuracy(double)** - validates and clamps before assigning (see Section 2.3).
- **setSymbol(char)** - directly assigns the new character with no validation required, since any character is a valid symbol.

2.6 Action Methods

Beyond getters and setters, the Typist class provides five action methods:

Method	What it does
typeCharacter()	Increments progress by 1. Only called when not burnt out.
slideBack(int amount)	Decrements progress; clamps to zero minimum.
burnOut(int turns)	Sets burntOut = true and stores the turn count.
recoverFromBurnout()	Decrements burnoutTurnsRemaining; clears burnout when counter rea
resetToStart()	Sets progress to 0 and fully clears all burnout state. Used before each

ches zero. race.

The recoverFromBurnout() method is the most non-trivial action method. It has no effect if the typist is not burnt out (guarded by an early return), decrements the counter by one, and then checks whether the counter has reached zero. When it does, both the counter and the boolean flag are cleared, returning the typist to a normal state automatically:

```
public void recoverFromBurnout()
{
    if (!burntOut) return;          // nothing to recover from
    burnoutTurnsRemaining--;
    if (burnoutTurnsRemaining <= 0)
    {
        burnoutTurnsRemaining = 0;
        burntOut = false;           // automatically recovered
```

}

}

3. TypingRace Class - Bug Analysis and Fixes

The TypingRace class was provided with several deliberate bugs. A thorough review identified seven issues, ranging from a crash-causing null pointer to inverted game logic. Each is documented below with the original faulty code, an explanation of its effect, and the corrected code.

Bug 1 - seat3Typist never reset before the race — startRace()

Problem: The original code only called resetToStart() on seats 1 and 2. Typist 3 was never reset, so if the race was run a second time (or if seat3 held any default non-zero state) that typist would start mid-track, giving them an unfair advantage.

Original (wrong):

```
seat1Typist.resetToStart();
seat2Typist.resetToStart();
// seat3Typist.resetToStart() — MISSING
```

Fixed:

```
seat1Typist.resetToStart();
seat2Typist.resetToStart();
seat3Typist.resetToStart();  // ADDED
```

Bug 2 - Win condition uses == instead of >= — raceFinishedBy()

Problem: The spec explicitly states the win condition must use >= because progress can overshoot passageLength. For example if a typist was at position 39 on a length-40 race and called typeCharacter() twice in one turn, their progress would become 41. With ==, this would never match 40, causing an infinite loop.

Original (wrong):

```
if (theTypist.getProgress() == passageLength)
```

Fixed:

```
return theTypist.getProgress() >= passageLength;
```

Bug 3 - Winner never announced (TODO left incomplete) startRace()

Problem: The original code had a comment marking the winner announcement as a TODO. It was never implemented. The fix determines which typist crossed the finish line first and prints their name and updated accuracy.

Original (wrong):

```
// TODO (Task 2a): Print the winner's name here
```

Fixed:

```
Typist winner = /* detect who finished */;
System.out.println("And the winner is... " + winner.getName());
System.out.printf("Final accuracy: %.2f%n", winner.getAccuracy());
```

Bug 4 - Mistype probability scales with accuracy - backwards logic advanceTypist()

Problem: With the original formula, a typist with accuracy 0.85 had a 0.85 x 0.30 = 25.5% mistype chance, while a typist with accuracy 0.30 had only a 9% chance. This is the opposite of the intended behaviour higher accuracy should mean fewer mistypes, not more. Using (1.0 - accuracy) corrects this.

Original (wrong):

```
if (Math.random() < theTypist.getAccuracy() * MISTYPE_BASE_CHANCE)
```

Fixed:

```
if (Math.random() < (1.0 - theTypist.getAccuracy()) * MISTYPE_BASE_CHANCE)
```

Bug 5 - spacesAfter can go negative at the finish

line printSeat()

Problem: When a typist reaches exactly passageLength, spacesAfter equals zero. The code then decrements it to -1 for burnt-out typists, breaking the lane display. A Math.max(0, ...) guard and a conditional decrement prevent this.

Original (wrong):

```
int spacesAfter = passageLength - theTypist.getProgress();
// spacesAfter-- without guard (becomes -1 when progress==passageLength)
```

Fixed:

```
int spacesAfter = Math.max(0, passageLength - progress);
if (spacesAfter > 0) { spacesAfter--; }
```

Bug 6 - Legend shows [zz] but code prints ~ after symbol printRace()

Problem: The legend told users to look for [zz] to identify a burnt-out typist, but printSeat() actually appends ~ after the typist's symbol (e.g. '①~'). This inconsistency would confuse anyone reading the race output.

Original (wrong):

```
"  [zz] = burnt out       [<] = just mistyped"
```

Fixed:

```
"  [~] = burnt out       [<] = just mistyped"
```

Bug 7 - Accuracy never updated after the race startRace()

Problem: The spec states that winning a race slightly increases a typist's accuracy and burning out slightly decreases it. Neither adjustment existed in the original code. The fix tracks burnout events during the race and applies +0.01 accuracy to the winner and -0.005 per burnout to affected typists.

Original (wrong):

```
// No accuracy adjustment anywhere in startRace()
```

Fixed:

```
winner.setAccuracy(winner.getAccuracy() + ACCURACY_WIN_BOOST);
applyBurnoutPenalty(seat1Typist, burnouts1); // all 3 seats
```

4. Testing and Validation

A dedicated test class, TypistTest.java, was written to verify every required behaviour of the Typist class. The suite runs 46 individual checks across 8 test groups. All 46 tests pass.

Test Group	Cases Covered	Result
TEST 1 — Constructor	All 6 fields initialise correctly	6 PASS
TEST 2 — typeCharacter	Progress advances by 1 each call	3 PASS
TEST 3 — slideBack	Normal slide; slide to exactly 0; slide below 0 (x2)	4 PASS
TEST 4 — burnOut/recover	State set; countdown turn by turn; auto-clear at 0; no-o when healthy	9 PASS
TEST 5 — resetToStart	Progress cleared; burntOut cleared; turnsRemaining cl ared	3 PASS
TEST 6 — setAccuracy	Valid value; boundary 0.0; boundary 1.0; below 0 clamp ed; above 1 clamp ed; extreme valid	9 PASS
TEST 7 — setSymbol	Symbol changes correctly	2 PASS
TEST 8 — Combined scenario	Full race interaction: type, slide, burnout, recover, reset	6 PASS
TOTAL		**46 / 46 PASS**

Terminal Output (Test Run)

The following output was produced by running the test suite:

```
==========================================
  TEST 1 - Constructor
==========================================
  PASS getName()     returns
  'TURBOFINGERS' PASS      getSymbol()
        returns      symbol      char      PASS
        getAccuracy() returns 0.85
  PASS getProgress()   starts   at   0
  PASS isBurntOut()    starts false
  PASS getBurnoutTurnsRemaining() starts at 0

==========================================
=======
TEST 3 - slideBack (cannot go below zero)
```

==========
=======

Progress before slideBack: 3

PASS slideBack(2) from 3 -> progress == 1

PASS slideBack(1) from 1 -> progress == 0

PASS slideBack(5) from 0 -> progress stays 0 (not -5)

PASS slideBack(10) from 1 -> progress clamped to 0

==========
===

TEST 4 - burnOut and recoverFromBurnout

==========
===

Burnt out for 3 turns

PASS After burnOut(3): isBurntOut() == true

PASS After burnOut(3): turnsRemaining == 3

After recoverFromBurnout() #1: turns = 2

PASS After recover #1: turnsRemaining == 2

PASS After recover #1: still burnt out After recoverFromBurnout() #2: turns = 1 PASS After recover #2: turnsRemaining == 1

After recoverFromBurnout() #3: turns = 0

PASS After recover #3: turnsRemaining == 0

PASS After recover #3: isBurntOut() == false

PASS recover when not burnt out: no effect

==========

TEST 6 - setAccuracy (clamping to 0.0 - 1.0)

==========

Set 0.75 -> result: 0.75 PASS

Set 0.0 -> result: 0.0 PASS

Set 1.0 -> result: 1.0 PASS Set -0.5 -> clamped to: 0.0 PASS Set 1.5 -> clamped to: 1.0 PASS

==========

RESULTS: 46 passed, 0 failed

==========

All tests passed!

5. Conclusion

This report has documented the full completion of Part I of the Typing Race Simulator project. The Typist class was designed and implemented from scratch following all specification requirements, with particular attention to encapsulation, validation, and boundary-safe state management.

The TypingRace class was thoroughly reviewed and seven bugs were identified and corrected. These ranged from a missing reset call that would give typist 3 an unfair advantage, to inverted mistype logic that penalised more accurate typists more heavily, to a win condition that could cause an infinite loop on overshoot.

A 46-test suite confirms that every required behaviour of the Typist class is correctly implemented, including all boundary conditions highlighted in the marking criteria: progress clamped at zero, burnout countdown, automatic recovery, resetToStart clearing all state, and accuracy clamping to [0.0, 1.0].

Files submitted: Typist.java | TypingRace.java | TypistTest.java | Report.pdf | README.md